

ВІННИЦЬКИЙ НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ

ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ ТА
КОМП'ЮТЕРНОЇ ІНЖЕНЕРІЇ

КАФЕДРА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

Звіт про виконання лабораторної роботи № 1

з дисципліни «Архітектура комп'ютера»

**на тему: «Методи переведення чисел з однієї системи числення в
іншу»**

Виконав (ла): ст. 2 курсу, групи 2ПІ-25Б

Коваль Олександр

Перевірив:.

1 МЕТА РОБОТИ

Практично засвоїти методи переведення чисел з однієї системи числення в іншу та набути навичок програмної реалізації переведення цілих і дробових чисел з двійкової системи числення у десяткову методом поліноміального розкладу з виведенням результату на екран.

2 ХІД РОБОТИ ТА ВИКОНАННЯ ЗАВДАННЯ

2.1 Постановка задачі

Розробити програмний застосунок (Desktop GUI), який переводить введене користувачем ціле або дробове число з двійкової системи числення у десяткову систему числення з виведенням результату на екран (завдання 2 методичних вказівок).

Правила роботи програми:

1. Користувач вводить двійкове число S (ціле або дробове), яке може містити знак «+» або «-».
2. Роздільником цілої та дробової частини може бути крапка «.» або кома «,» (наприклад, 1101.101 або 1101,101).
3. Програма виконує валідацію введення: дозволені лише символи «0», «1», знак та один роздільник; у разі помилки виводиться інформативне повідомлення.
4. Обчислення виконується методом поліноміального розкладу за степенями двійки з використанням модуля decimal (висока точність без похибок IEEE-754).

2.2 Виділення трьох основних ділянок алгоритму

Відповідно до теоретичних положень роботи, розроблений алгоритм містить усі три фундаментальні типи керуючих структур.

2.2.1 Лінійна ділянка (послідовне виконання дій)

У програмі наявні дві лінійні підділянки:

- Початкова лінійна ділянка: зчитування вхідного рядка S ; нормалізація (видалення пробілів, заміна коми на крапку); ініціалізація накопичувачів $D_int = 0$, $D_frac = 0$.

- Фінальна лінійна ділянка: підсумовування $D = D_int + D_frac$; врахування знаку (якщо $sign == '-'$, то $D = -D$); формування поліноміальної формули, покрокової таблиці розрядів та виведення результату на екран.

2.2.2 Циклічна ділянка (повторення операцій)

- реалізована двома циклами з лічильником: $for\ i = 0..N-1$ (обробка розрядів цілої частини) та $for\ j = 1..M$ (обробка розрядів дробової частини);
- повторює обчислення внеску кожного двійкового розряду: $D += bit \cdot 2^{power}$;
- на кожній ітерації: визначається степінь $power$ ($N-1-i$ для цілої, $-j$ для дробової частини); обчислюється вага розряду 2^{power} ; накопичується сума та записується член полінома.

2.2.3 Розгалужена ділянка (прийняття рішень, умовні оператори)

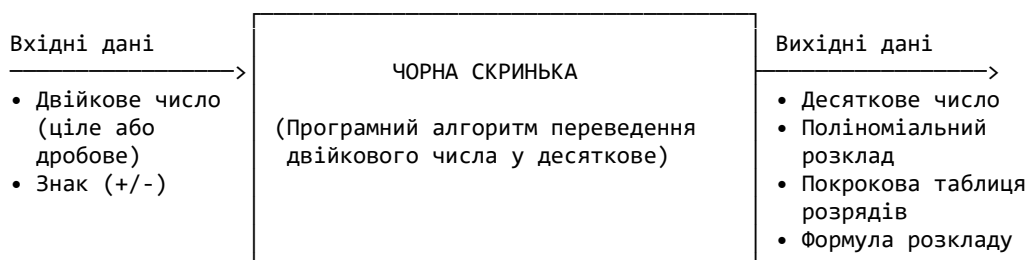
- Розгалуження 1 (валідація): Якщо рядок містить символи, відмінні від «0», «1», «.», «+», «-» $\Rightarrow$ вивести помилку «Некоректне число»; інакше продовжити обчислення.
- Розгалуження 2 (знак): Якщо $sign == '-' \Rightarrow D = -(D_int + D_frac)$; інакше $D = D_int + D_frac$.

2.3 Представлення алгоритму в трьох виглядах

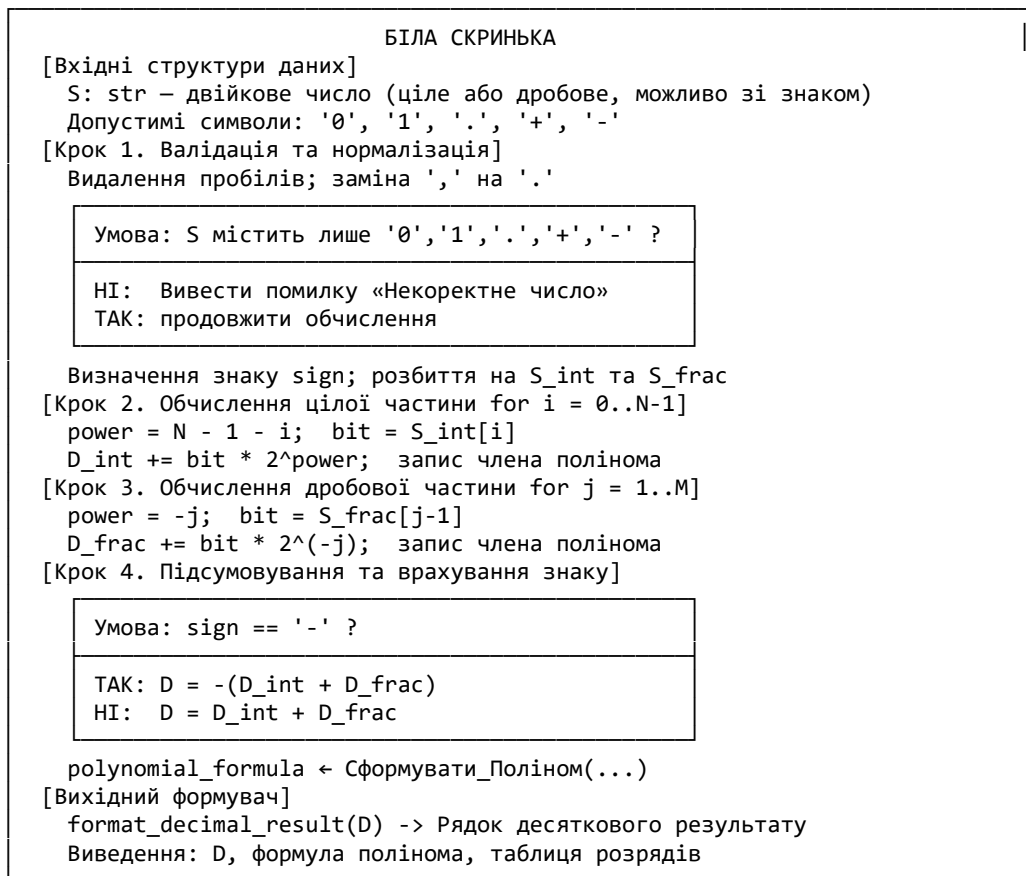
Згідно з вимогами лабораторної роботи (пункт 2 ходу роботи), алгоритм подано у трьох стандартизованих формах.

Вигляд 1. Модель «чорної» та «білої» скриньки

Модель «чорної скриньки» відображає відношення між входом і виходом системи без розкриття внутрішньої реалізації:



Модель «білої скриньки» відображає внутрішню будову системи, структури даних, правила обчислення та взаємозв'язки:



Вигляд 2. Графічна блок-схема (Flowchart)

Блок-схему побудовано з дотриманням правил Таблиці 1 методичних вказівок (термінатори «Початок/Кінець», паралелограми вводу/виводу, прямокутники обробки, ромби умов). Файл векторної схеми згенеровано автоматично (algorithm_flowchart.svg). Блок-схему наведено на рисунку 2.1.

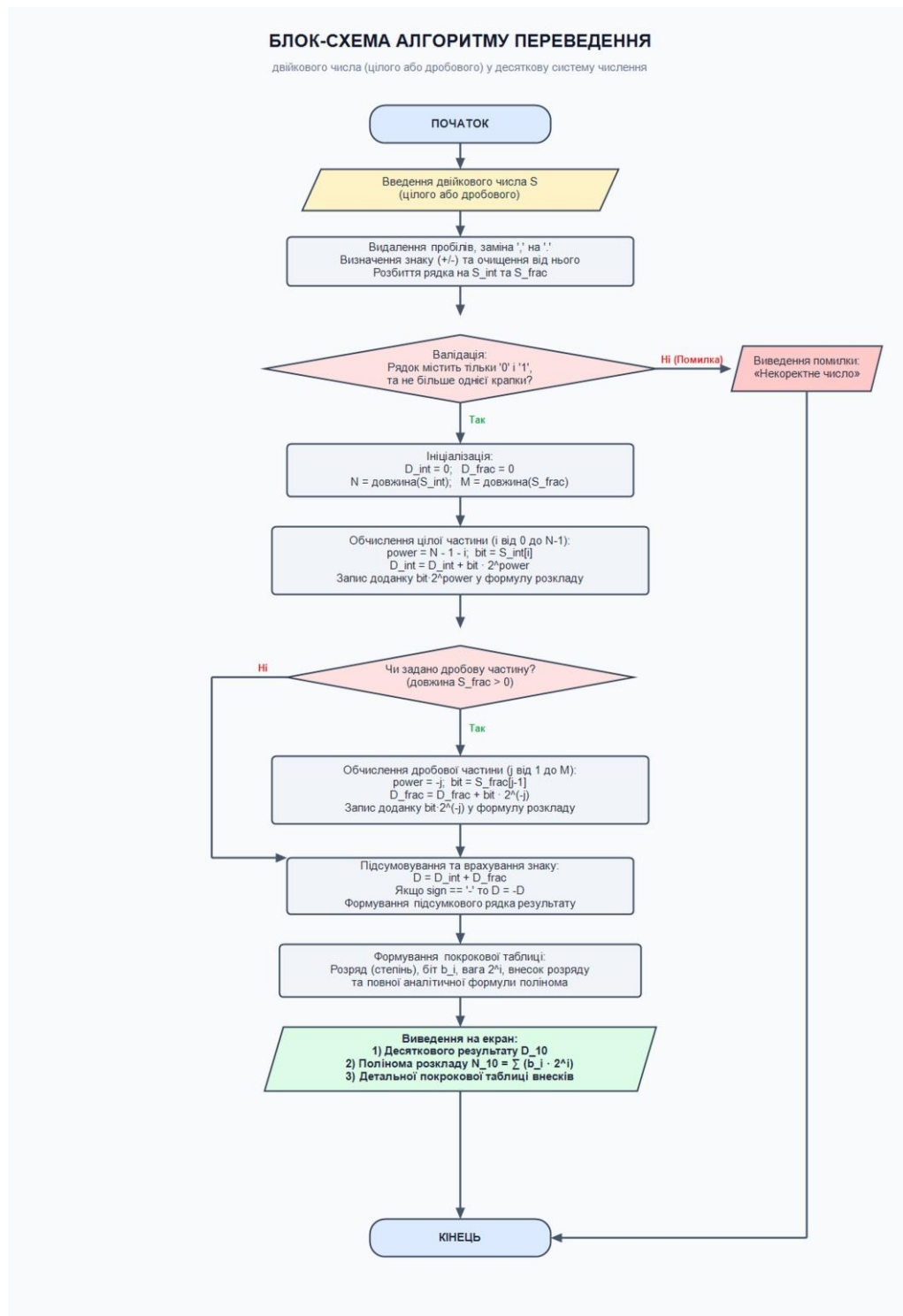


Рисунок 2.1 – Блок-схема алгоритму переведення двійкового числа у десяткову систему числення

Вигляд 3. Структурований описовий алгоритм (псевдокод)

АЛГОРИТМ Переведення_Двійкового_У_Десяткове:

ВХІД:

S (рядок: двійкове число, можливо зі знаком '+/-')
та роздільником '.' або ',' (ціле або дробове)

Допустимі символи: '0', '1', '.', '+', '-'

ВИХІД:

D (десятькове значення), `polynomial_formula`, таблиця розрядів

КРОК 1 [ВАЛІДАЦІЯ ТА НОРМАЛІЗАЦІЯ]:

S ← видалити пробіли(S); S ← замінити кома на крапку(S)
ЯКЩО S містить символи, відмінні від '0', '1', '.', '+', '-' ТОДІ:
ВІВЕСТИ помилку «Некоректне число»; КІНЕЦЬ
КІНЕЦЬ УМОВИ; sign ← знак числа; Розбити S на S_int та S_frac

КРОК 2 [ОБЧИСЛЕННЯ ЦІЛОЇ ЧАСТИНИ]:

ДЛЯ i ВІД 0 ДО N-1 ВИКОНАТИ:
power ← N - 1 - i; bit ← S_int[i]
D_int ← D_int + bit · 2^{power}; запис члена полінома

КРОК 2.1 [ОБЧИСЛЕННЯ ДРОБОВОЇ ЧАСТИНИ]:

ДЛЯ j ВІД 1 ДО M ВИКОНАТИ:
power ← -j; bit ← S_frac[j-1]
D_frac ← D_frac + bit · 2^(-j); запис члена полінома
КІНЕЦЬ ЦИКЛУ
КІНЕЦЬ ЦИКЛУ

КРОК 3 [ПІДСУМОВУВАННЯ ТА ВРАХУВАННЯ ЗНАКУ]:

D ← D_int + D_frac
ЯКЩО sign == '-' ТОДІ: D ← -D

КІНЕЦЬ УМОВИ

КРОК 4 [ФОРМУВАННЯ РЕЗУЛЬТАТУ]:

`polynomial_formula` ← Сформувати_Поліном(...)
таблиця_розрядів ← Сформувати_Таблицю_Розрядів(...)
ВІВЕСТИ D, `polynomial_formula`, таблиця_розрядів
КІНЕЦЬ АЛГОРИТМУ

2.4 Програмна реалізація

Проект реалізовано кількома файлами згідно з принципом модульності:

- `converter.py` — ядро математичної логіки: валідація, нормалізація та поліноміальний розклад з використанням модуля `decimal`;
- `main.py` — повноцінний настільний графічний застосунок (Desktop GUI на `tkinter`) з полем введення, швидкими пресетами, покроковою таблицею розрядів та копіюванням результату;
- `test_converter.py` — модульне тестування всіх сценаріїв переведення (цілі, дробові, від'ємні числа та некоректні введення);
- `generate_flowchart.py` — генератор векторної блок-схеми у форматі SVG (ГОСТ 19.701-90).

Головна функція `convert_binary_to_decimal()` повертає десятковий результат, аналітичну поліноміальну формулу та покрокові внески розрядів.
Запуск застосунку: `python main.py`.

2.5 Результати тестування та верифікація

Для верифікації правильності роботи програми протестовано всі основні сценарії: цілі, дробові та від'ємні числа, кому як роздільник, нормалізацію ведучих нулів та некоректні введення (test_converter.py). Результати тестування наведено в таблиці 2.1.

Таблиця 2.1 – Результати тестування програми переведення чисел

№	Сценарій	Вхідне число	Очікуваний результат	Отриманий результат	Статус	Точність	Примітка
1	Приклад з методички	110100101	421	421	Пройдено	exact	—
2	Ціле число	1111	15	15	Пройдено	exact	—
3	Дробове число	1011.1101	11.8125	11.8125	Пройдено	exact	—
4	Кома як роздільник	1101,101	13.625	13.625	Пройдено	exact	—
5	Від'ємне число	-101.011	-5.375	-5.375	Пройдено	exact	—
6	Некоректне введення	1021	Помилка	Помилка	Пройдено	—	—

Приклад виконання програми (переведення числа 110100101₂):

```
=====
  КОНВЕРТЕР СИСТЕМ ЧИСЛЕННЯ (Лаб. роб. №1)
  Переведення двійкового числа у десяткове
=====
Введено двійкове число: 110100101
-----
Математичний розклад за поліномом:
N10 = 1·28 + 1·27 + 0·26 + 1·25 + 0·24
      + 0·23 + 1·22 + 0·21 + 1·20
      = 256 + 128 + 0 + 32 + 0 + 0 + 4 + 0 + 1
=====
РЕЗУЛЬТАТ: 42110
=====
Покрокова таблиця розрядів:
Частина | Степінь | Біт | Вага 2i | Внесок
Ціла    | 28    | 1  | 256      | 256
Ціла    | 27    | 1  | 128      | 128
Ціла    | 26    | 0  | 64       | 0
Ціла    | 25    | 1  | 32       | 32
Ціла    | 24    | 0  | 16       | 0
Ціла    | 23    | 0  | 8        | 0
Ціла    | 22    | 1  | 4        | 4
Ціла    | 21    | 0  | 2        | 0
Ціла    | 20    | 1  | 1        | 1
```

Усі 6 автоматизованих тестів виконано без помилок.

3 ЛІСТИНГ ПРОГРАМИ

3.1 Модуль converter.py (повний лістинг)

```
"""
converter.py
Модуль для переведення двійкових чисел (цілих та дробових) у десяткову систему числення.
Підтримує точні обчислення за допомогою модуля decimal.
"""

from decimal import Decimal, getcontext
from typing import NamedTuple, List, Tuple

# Встановлюємо достатню точність для роботи з довгими дробами
getcontext().prec = 50

class BitContribution(NamedTuple):
    index: int          # Позиція степеня (наприклад: 3, 2, 1, 0 для цілої, -1, -2 для дробової)
    bit: str             # Двійковий символ ('0' або '1')
    power_value: Decimal # Значення 2^index
    contribution: Decimal # bit * 2^index

class ConversionResult(NamedTuple):
    original_input: str
    is_negative: bool
    integer_part_binary: str
    fractional_part_binary: str
    integer_contributions: List[BitContribution]
    fractional_contributions: List[BitContribution]
    integer_decimal: Decimal
    fractional_decimal: Decimal
    total_decimal: Decimal
    polynomial_formula: str
    polynomial_with_values: str

def validate_and_normalize(binary_str: str) -> Tuple[bool, str, str]:
    """
    Перевіряє коректність введеного двійкового числа та нормалізує його.
    Повертає: (чи_валідне, повідомлення_про_помилку, нормалізований_рядок)
    """
    if not binary_str:
        return False, "Помилка: рядок введення порожній. Будь ласка, введіть число.", ""

    s = binary_str.strip().replace(" ", "")
    if not s:
        return False, "Помилка: введено лише пробіли.", ""

    # Заміна коми на крапку для підтримки обох роздільників
    s = s.replace(",", ".")

    # Обробка знаку
    sign = ""
    if s[0] in ("+", "-"):
        sign = s[0]
        s = s[1:]

    if not s:
        return False, "Помилка: після знаку відсутні цифри.", ""
```

```

# Перевірка кількості крапок
dot_count = s.count(".")
if dot_count > 1:
    return False, "Помилка: число містить більше однієї крапки/коми.", ""

# Перевірка на неприпустимі символи
valid_chars = {"0", "1", "."}
invalid_chars = set(s) - valid_chars
if invalid_chars:
    return False, f"Помилка: двійкове число може містити лише 0 та 1. Знайдено неприпустимі символи: {' '.join(sorted(invalid_chars))}", ""

# Якщо крапка є єдиним символом
if s == ".":
    return False, "Помилка: введено лише розділовий знак без цифр.", ""

# Нормалізація вигляду: наприклад, .101 -> 0.101, 110. -> 110.0
if "." in s:
    parts = s.split(".")
    int_p = parts[0] if parts[0] else "0"
    frac_p = parts[1] if parts[1] else "0"
    normalized = f"{sign}{int_p}.{frac_p}"
else:
    normalized = f"{sign}{s}"

return True, "", normalized

def convert_binary_to_decimal(binary_str: str) -> ConversionResult:
    """
    Виконує переведення двійкового числа у десяткове з покроковим розкладом за поліномом.
    Викликає ValueError у разі некоректного введення.
    """
    is_valid, err_msg, normalized = validate_and_normalize(binary_str)
    if not is_valid:
        raise ValueError(err_msg)

    is_negative = normalized.startswith("-")
    clean_num = normalized.lstrip("+ -")

    if "." in clean_num:
        int_str, frac_str = clean_num.split(".")
    else:
        int_str, frac_str = clean_num, ""

    # Видалення зайвих ведучих нулів у цілій частині (залишаємо хоча б один '0')
    int_str_stripped = int_str.lstrip("0")
    if not int_str_stripped:
        int_str = "0"
    else:
        int_str = int_str_stripped

    # Видалення зайвих нулів у кінці дробової частини
    frac_str = frac_str.rstrip("0")

    # 1. Обчислення цілої частини:  $\sum_{i=0..n-1} a_i \cdot 2^i$ 
    int_contributions: List[BitContribution] = []
    int_decimal = Decimal(0)
    n = len(int_str)

    for idx, char in enumerate(int_str):
        power = n - 1 - idx
        power_val = Decimal(2) ** power

```

```

bit_val = Decimal(1) if char == "1" else Decimal(0)
contrib = bit_val * power_val
int_contributions.append(BitContribution(
    index=power,
    bit=char,
    power_value=power_val,
    contribution=contrib
))
int_decimal += contrib

# 2. Обчислення дробової частини:  $\sum_{j=1..m} b_j \cdot 2^{-j}$ 
frac_contributions: List[BitContribution] = []
frac_decimal = Decimal(0)

for j, char in enumerate(frac_str, start=1):
    power = -j
    power_val = Decimal(2) ** power
    bit_val = Decimal(1) if char == "1" else Decimal(0)
    contrib = bit_val * power_val
    frac_contributions.append(BitContribution(
        index=power,
        bit=char,
        power_value=power_val,
        contribution=contrib
    ))
    frac_decimal += contrib

# Підсумкове значення
total_abs = int_decimal + frac_decimal
total_decimal = -total_abs if is_negative else total_abs

# 3. Формування математичного розкладу за поліномом
poly_terms = []
poly_calc_terms = []

# Додаємо цілі розряди
for c in int_contributions:
    poly_terms.append(f"{c.bit}·2^{{{c.index}}}")
    poly_calc_terms.append(f"{c.contribution}")

# Додаємо дробові розряди
for c in frac_contributions:
    poly_terms.append(f"{c.bit}·2^{{{c.index}}}")
    poly_calc_terms.append(f"{c.contribution}")

if not poly_terms:
    polynomial_formula = "0·2^{0}"
    polynomial_with_values = "0"
else:
    polynomial_formula = " + ".join(poly_terms)
    polynomial_with_values = " + ".join(poly_calc_terms)

if is_negative:
    polynomial_formula = f"-({polynomial_formula})"
    polynomial_with_values = f"-({polynomial_with_values})"

return ConversionResult(
    original_input=binary_str,
    is_negative=is_negative,
    integer_part_binary=int_str,
    fractional_part_binary=frac_str,
    integer_contributions=int_contributions,
    fractional_contributions=frac_contributions,
    integer_decimal=int_decimal,
    fractional_decimal=frac_decimal,

```

```

        total_decimal=total_decimal,
        polynomial_formula=polynomial_formula,
        polynomial_with_values=polynomial_with_values
    )

def format_decimal_result(val: Decimal) -> str:
    """
    Красиво форматувє Decimal у рядок без експоненційного запису та без зайвих нулів у кінці.
    """
    s = f"{val:f}"
    if "." in s:
        s = s.rstrip("0").rstrip(".")
    return s if s else "0"

```

3.2 Модуль main.py (повний лістинг)

```

"""
main.py
Графічний інтерфейс користувача (GUI) для Лабораторної роботи №1.
Тема: Методи переведення чисел з однієї системи числення в іншу.
Завдання 2: Переведення цілих або дробових чисел з двійкової у десяткову систему числення.
"""

import tkinter as tk
from tkinter import ttk, messagebox
from decimal import Decimal
from converter import convert_binary_to_decimal, format_decimal_result, ConversionResult

class BinaryDecimalConverterApp:
    def __init__(self, root: tk.Tk):
        self.root = root
        self.root.title("Лабораторна робота №1 | Конвертер двійкових чисел у десяткові")
        self.root.geometry("820x720")
        self.root.minsize(700, 600)

        # Налаштування кольорової схеми та стилів
        self.bg_color = "#f8fafc"
        self.card_bg = "#ffffff"
        self.primary_color = "#2563eb"
        self.primary_hover = "#1d4ed8"
        self.secondary_color = "#64748b"
        self.text_color = "#1e293b"
        self.text_muted = "#64748b"
        self.border_color = "#cbd5e1"
        self.success_color = "#16a34a"
        self.error_color = "#dc2626"

        self.root.configure(bg=self.bg_color)
        self.setup_styles()
        self.create_widgets()

    def setup_styles(self):
        self.style = ttk.Style()
        try:
            self.style.theme_use("clam")
        except Exception:
            pass

        # Загальні стилі віджетів ttk

```

```

        self.style.configure(".", background=self.bg_color, foreground=self.text_color,
font=("Segoe UI", 10))
        self.style.configure("Card.TFrame", background=self.card_bg, relief="flat")
        self.style.configure("Header.TLabel", font=("Segoe UI", 15, "bold"),
foreground="#0f172a", background=self.card_bg)
        self.style.configure("SubHeader.TLabel", font=("Segoe UI", 9),
foreground=self.text_muted, background=self.card_bg)
        self.style.configure("FieldLabel.TLabel", font=("Segoe UI", 10, "bold"),
foreground=self.text_color, background=self.card_bg)
        self.style.configure("ResultLabel.TLabel", font=("Segoe UI", 11, "bold"),
foreground="#0f172a", background=self.card_bg)
        self.style.configure("Formula.TLabel", font=("Consolas", 10), foreground="#0369a1",
background="#f0f9ff")

# Стиль таблиці Treeview
self.style.configure(
    "Treeview",
    background="ffffff",
    foreground=self.text_color,
    fieldbackground="ffffff",
    rowheight=26,
    font=("Segoe UI", 9)
)
self.style.configure(
    "Treeview.Heading",
    font=("Segoe UI", 9, "bold"),
    background="#e2e8f0",
    foreground="#334155"
)
self.style.map("Treeview", background=[("selected", "#dbeafe")], foreground=[("selected",
"#1e3a8a")])

def create_widgets(self):
    # Головний скрольований контейнер
    main_frame = tk.Frame(self.root, bg=self.bg_color)
    main_frame.pack(fill=tk.BOTH, expand=True, padx=20, pady=16)

    # ===== 1. ШАПКА =====
    header_card = tk.Frame(main_frame, bg=self.card_bg, bd=1, relief=tk.SOLID,
highlightbackground=self.border_color)
    header_card.pack(fill=tk.X, pady=(0, 12), ipady=8, ipadx=14)

    title_lbl = ttk.Label(
        header_card,
        text="Калькулятор переведення з двійкової у десяткову систему",
        style="Header.TLabel"
    )
    title_lbl.pack(anchor="w")

    subtitle_lbl = ttk.Label(
        header_card,
        text="Лабораторна робота №1 з дисципліни «Архітектура комп'ютера» | Метод розкладу за
поліномом степенів 2",
        style="SubHeader.TLabel"
    )
    subtitle_lbl.pack(anchor="w", pady=(2, 0))

    # ===== 2. БЛОК ВВЕДЕННЯ =====
    input_card = tk.Frame(main_frame, bg=self.card_bg, bd=1, relief=tk.SOLID,
highlightbackground=self.border_color)
    input_card.pack(fill=tk.X, pady=(0, 12), padx=0, ipady=10, ipadx=14)

    input_title = ttk.Label(
        input_card,
        text="Введіть двійкове число (ціле або дробове):",
        style="FieldLabel.TLabel"
    )

```

```

)
input_title.pack(anchor="w")

# Поле вводу та кнопки
entry_row = tk.Frame(input_card, bg=self.card_bg)
entry_row.pack(fill=tk.X, pady=(8, 8))

self.input_var = tk.StringVar(value="110100101")
self.entry = tk.Entry(
    entry_row,
    textvariable=self.input_var,
    font=("Consolas", 13),
    bd=1,
    relief=tk.SOLID,
    highlightthickness=1,
    highlightcolor=self.primary_color,
    highlightbackground=self.border_color,
    fg="#0f172a",
    bg="ffffff"
)
self.entry.pack(side=tk.LEFT, fill=tk.X, expand=True, ipady=6, padx=(0, 8))
self.entry.bind("<Return>", lambda event: self.calculate())
self.entry.focus_set()

# Кнопка переведення
calc_btn = tk.Button(
    entry_row,
    text="Перевести →",
    command=self.calculate,
    bg=self.primary_color,
    fg="ffffff",
    activebackground=self.primary_hover,
    activeforeground="ffffff",
    font=("Segoe UI", 10, "bold"),
    relief=tk.FLAT,
    cursor="hand2",
    padx=16,
    pady=6
)
calc_btn.pack(side=tk.LEFT, padx=(0, 6))

# Кнопка очищення
clear_btn = tk.Button(
    entry_row,
    text="Очистити",
    command=self.clear_input,
    bg="#f1f5f9",
    fg="#475569",
    activebackground="#e2e8f0",
    activeforeground="#1e293b",
    font=("Segoe UI", 10),
    relief=tk.FLAT,
    cursor="hand2",
    padx=12,
    pady=6
)
clear_btn.pack(side=tk.LEFT)

# Рядок швидких прикладів
presets_row = tk.Frame(input_card, bg=self.card_bg)
presets_row.pack(fill=tk.X, pady=(4, 0))

preset_lbl = tk.Label(presets_row, text="Швидкі приклади:", bg=self.card_bg,
fg=self.text_muted, font=("Segoe UI", 8))
preset_lbl.pack(side=tk.LEFT, padx=(0, 8))

```

```

samples = [
    ("110100101 (Методичка)", "110100101"),
    ("1011.1101 (Дріб)", "1011.1101"),
    ("101.011 (Від'ємне)", "-101.011"),
    ("0.0011 (Малий дріб)", "0.0011"),
    ("11111111 (255)", "11111111")
]
for title, val in samples:
    btn = tk.Button(
        presets_row,
        text=title,
        command=lambda v=val: self.load_preset(v),
        bg="#f8fafc",
        fg="#2563eb",
        activebackground="#e0f2fe",
        font=("Segoe UI", 8),
        relief=tk.FLAT,
        cursor="hand2",
        padx=6,
        pady=2
    )
    btn.pack(side=tk.LEFT, padx=3)

# ===== 3. БЛОК РЕЗУЛЬТАТУ =====
result_card = tk.Frame(main_frame, bg=self.card_bg, bd=1, relief=tk.SOLID,
highlightbackground=self.border_color)
result_card.pack(fill=tk.X, pady=(0, 12), ipady=10, ipadx=14)

res_header_row = tk.Frame(result_card, bg=self.card_bg)
res_header_row.pack(fill=tk.X)

res_title = ttk.Label(res_header_row, text="Результат переведення:",
style="FieldLabel.TLabel")
res_title.pack(side=tk.LEFT)

self.copy_btn = tk.Button(
    res_header_row,
    text="📄 Скопіювати результат",
    command=self.copy_result,
    bg="#f8fafc",
    fg="#0f172a",
    activebackground="#e2e8f0",
    font=("Segoe UI", 9),
    relief=tk.FLAT,
    cursor="hand2",
    padx=8,
    pady=2
)
self.copy_btn.pack(side=tk.RIGHT)

# Головне поле виведення результату
self.result_display_var = tk.StringVar(value="Введіть число та натисніть «Перевести»")
self.result_label = tk.Label(
    result_card,
    textvariable=self.result_display_var,
    font=("Segoe UI", 16, "bold"),
    bg="#f0fdf4",
    fg="#15803d",
    padx=12,
    pady=8,
    anchor="w",
    relief=tk.SOLID,
    bd=1
)
self.result_label.pack(fill=tk.X, pady=(8, 8))

```

```

# Формула розкладу за поліномом
poly_title = tk.Label(result_card, text="Математичний розклад за поліномом:",
bg=self.card_bg, fg=self.text_muted, font=("Segoe UI", 9, "bold"))
poly_title.pack(anchor="w", pady=(2, 2))

self.formula_text = tk.Text(
    result_card,
    height=2,
    font=("Consolas", 10),
    bg="#f8fafc",
    fg="#0369a1",
    bd=1,
    relief=tk.SOLID,
    wrap=tk.WORD,
    padx=8,
    pady=6
)
self.formula_text.pack(fill=tk.X)
self.formula_text.insert(tk.END, " $N_{10} = \sum (b_i \cdot 2^i)$ ")
self.formula_text.config(state=tk.DISABLED)

# ===== 4. ПОКРОКОВА ТАБЛИЦЯ РОЗКЛАДУ =====
table_card = tk.Frame(main_frame, bg=self.card_bg, bd=1, relief=tk.SOLID,
highlightbackground=self.border_color)
table_card.pack(fill=tk.BOTH, expand=True, pady=(0, 8), ipady=8, ipadx=14)

table_title = ttk.Label(table_card, text="Покрокова деталізація внеску кожного розряду:",
style="FieldLabel.TLabel")
table_title.pack(anchor="w", pady=(0, 6))

columns = ("part", "pos", "bit", "power_formula", "power_val", "contrib")
self.tree = ttk.Treeview(table_card, columns=columns, show="headings", height=8)

self.tree.heading("part", text="Частина числа")
self.tree.heading("pos", text="Степінь (i)")
self.tree.heading("bit", text="Біт (bi)")
self.tree.heading("power_formula", text="Степінь двійки")
self.tree.heading("power_val", text="Вара розряду (2i)")
self.tree.heading("contrib", text="Внесок (bi · 2i)")

self.tree.column("part", width=120, anchor="center")
self.tree.column("pos", width=90, anchor="center")
self.tree.column("bit", width=80, anchor="center")
self.tree.column("power_formula", width=110, anchor="center")
self.tree.column("power_val", width=150, anchor="e")
self.tree.column("contrib", width=150, anchor="e")

tree_scroll = ttk.Scrollbar(table_card, orient="vertical", command=self.tree.yview)
self.tree.configure(yscrollcommand=tree_scroll.set)

self.tree.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
tree_scroll.pack(side=tk.RIGHT, fill=tk.Y)

# ===== 5. СТАТУСНИЙ РЯДОК =====
self.status_var = tk.StringVar(value="Готово до роботи.")
status_bar = tk.Label(
    self.root,
    textvariable=self.status_var,
    font=("Segoe UI", 9),
    bg="#e2e8f0",
    fg="#475569",
    anchor="w",
    padx=12,
    pady=4
)

```

```

status_bar.pack(fill=tk.X, side=tk.BOTTOM)

# Розраховуємо початкове значення
self.calculate()

def load_preset(self, val: str):
    self.input_var.set(val)
    self.calculate()

def clear_input(self):
    self.input_var.set("")
    self.result_display_var.set("Введіть число та натисніть «Перевести»")
    self.result_label.configure(bg="#f8fafc", fg=self.text_color)
    self.formula_text.config(state=tk.NORMAL)
    self.formula_text.delete("1.0", tk.END)
    self.formula_text.insert(tk.END, " $N_{10} = \sum (b_i \cdot 2^i)$ ")
    self.formula_text.config(state=tk.DISABLED)
    for row in self.tree.get_children():
        self.tree.delete(row)
    self.status_var.set("Поле очищено.")
    self.entry.focus_set()

def copy_result(self):
    res_text = self.result_display_var.get()
    if res_text and not res_text.startswith("Помилка") and not
res_text.startswith("Введіть"):
        # Витягуємо тільки числовий результат
        clean_res = res_text.replace("Результат:", "").replace("_10", "").strip()
        self.root.clipboard_clear()
        self.root.clipboard_append(clean_res)
        self.status_var.set(f"Результат {clean_res} скопійовано в буфер обміну! ✓")
    else:
        self.status_var.set("Немає результату для копіювання.")

def calculate(self):
    val = self.input_var.get().strip()
    if not val:
        self.result_display_var.set("Помилка: введіть двійкове число!")
        self.result_label.configure(bg="#fef2f2", fg=self.error_color)
        self.status_var.set("Поле введення порожнє.")
        return

    try:
        res: ConversionResult = convert_binary_to_decimal(val)
    except ValueError as ex:
        self.result_display_var.set(f"{ex}")
        self.result_label.configure(bg="#fef2f2", fg=self.error_color)
        self.status_var.set(f"Помилка: {ex}")
        # Очищуємо формулу та таблицю
        self.formula_text.config(state=tk.NORMAL)
        self.formula_text.delete("1.0", tk.END)
        self.formula_text.insert(tk.END, "Помилка валідації вхідних даних.")
        self.formula_text.config(state=tk.DISABLED)
        for row in self.tree.get_children():
            self.tree.delete(row)
        return

    # Відображення успішного результату
    formatted_dec = format_decimal_result(res.total_decimal)
    self.result_display_var.set(f"Результат: {formatted_dec}_{10} (з {val}_{2})")
    self.result_label.configure(bg="#f0fdf4", fg="#15803d")

    # Оновлення формули
    self.formula_text.config(state=tk.NORMAL)
    self.formula_text.delete("1.0", tk.END)

```

```

self.formula_text.insert(
    tk.END,
    f"Формула: {res.polynomial_formula}\n"
    f"Значення: {res.polynomial_with_values} = {formatted_dec}_10"
)
self.formula_text.config(state=tk.DISABLED)

# Оновлення таблиці
for row in self.tree.get_children():
    self.tree.delete(row)

for c in res.integer_contributions:
    self.tree.insert("", tk.END, values=(
        "Ціла частина",
        f"2^{c.index}",
        c.bit,
        f"2^{c.index}",
        format_decimal_result(c.power_value),
        format_decimal_result(c.contribution)
    ))

for c in res.fractional_contributions:
    self.tree.insert("", tk.END, values=(
        "Дробова частина",
        f"2^{c.index}",
        c.bit,
        f"2^{c.index} (1/2^{abs(c.index)})",
        format_decimal_result(c.power_value),
        format_decimal_result(c.contribution)
    ))

self.status_var.set(f"Успішно переведено {val}_2 = {formatted_dec}_10 ✓")

def main():
    root = tk.Tk()
    app = BinaryDecimalConverterApp(root)
    root.mainloop()

if __name__ == "__main__":
    main()

```

4 ВІДПОВІДІ НА КОНТРОЛЬНІ ПИТАННЯ ТА ЗАВДАННЯ

1. Що таке система числення?

Система числення — це спосіб запису (зображення) чисел за допомогою певного набору символів (цифр) та правил виконання арифметичних операцій над ними.

2. Які системи числення називаються позиційними?

Позиційними називаються системи числення, в яких значення (вага) кожної цифри залежить від її позиції (розряду) в записі числа. Приклади: двійкова, вісімкова, десяткова, шістнадцяткова.

3. Які системи числення використовуються в обчислювальній техніці?

В обчислювальній техніці використовуються двійкова (основа 2), вісімкова (основа 8) та шістнадцяткова (основа 16) системи числення, оскільки вони пов'язані з двійковою степенями числа 2.

4. Що таке основа системи числення?

Основа системи числення — це кількість цифр, що використовуються для запису чисел у даній системі. Наприклад, основа двійкової системи дорівнює 2 (цифри 0 і 1), десятикової — 10.

5. Як перевести ціле число з десятикової системи числення у двійкову?

Ціле число послідовно ділиться на основу нової системи (2) до отримання нульової частки; число в новій системі записується із залишків від ділення, починаючи з останнього.

6. Як перевести число з будь-якої системи числення у десятикову?

Число представляється у вигляді полінома за степенями основи, і всі члени полінома розкриваються (обчислюються) у десятиковій системі числення.

7. Що таке поліноміальний розклад числа?

Поліноміальний розклад — це представлення числа у вигляді суми добутків цифр на вагу відповідних розрядів: $N = \sum a_i \cdot p^i$, де p — основа системи, a_i — цифра розряду.

8. Як перевести дробове число з двійкової системи числення у десяткову?

Дробова частина розкладається за від'ємними степенями двійки: $0,1101_2 = 1 \cdot 2^{-1} + 1 \cdot 2^{-2} + 0 \cdot 2^{-3} + 1 \cdot 2^{-4} = 0,5 + 0,25 + 0 + 0,0625 = 0,8125_{10}$.

9. Що таке вага розряду?

Вага розряду — це значення степеня основи, що відповідає позиції цифри в записі числа. Для цілої частини вага дорівнює p^i ($i \geq 0$), для дробової — p^{-j} ($j \geq 1$).

10. Які методи переведення чисел з однієї системи числення в іншу існують?

Існують два основних методи: табличний (з використанням таблиць відповідності чисел) та розрахунковий (виконання арифметичних операцій ділення або поліноміального розкладу).

11. У чому суть табличного методу переведення?

Табличний метод заснований на використанні таблиць відповідності чисел різних систем числення. Він громіздкий і потребує значного об'єму пам'яті, але може застосовуватись для будь-яких систем.

12. Чому в ЕОМ використовується двійкова система числення?

Двійкова система використовується тому, що технічно найпростіше реалізувати два стійкі стани (0 і 1), а також тому, що двійкова арифметика є найпростішою для апаратної реалізації.

13. Як перевести число з двійкової системи числення у вісімкову?

Двійкове число розбивається на тріади (групи по три розряди) вліво і вправо від коми, кожна тріада замінюється відповідною вісімковою цифрою.

14. Як перевести число з двійкової системи числення у шістнадцяткову?

Двійкове число розбивається на тетради (групи по чотири розряди), кожна тетрада замінюється відповідною шістнадцятковою цифрою (0–9, A–F).

15. Як позначається система числення в записі числа?

Система числення позначається нижнім індексом біля числа, наприклад: 1101_2 — двійкова, 735_8 — вісімкова, $92C_{16}$ — шістнадцяткова, 421_{10} — десяткова.

16. Які особливості переведення від'ємних чисел?

Для від'ємного числа спочатку переводиться його модуль, після чого до результату застосовується знак «–». У програмі знак визначається на етапі валідації та враховується при підсумовуванні.

5 ВИСНОВКИ

Під час виконання лабораторної роботи №1:

1. Засвоєно методи переведення чисел з однієї системи числення в іншу, зокрема метод поліноміального розкладу для переведення цілих і дробових двійкових чисел у десяткову систему числення.
2. Розроблено програмний застосунок (Desktop GUI на Tkinter), який переводить двійкові числа у десяткові з виведенням результату, аналітичної формули розкладу та покрокової таблиці розрядів.
3. Алгоритм представлено у трьох стандартизованих виглядах: моделі «чорної» та «білої» скриньки, графічній блок-схемі (ГОСТ 19.701-90) та структурованому псевдокоді.
4. Використано модуль decimal, що забезпечує високу точність обчислень та усуває похибки двійкової арифметики з плаваючою крапкою (IEEE-754).
5. Проведено модульне тестування всіх сценаріїв (цілі, дробові, від'ємні числа, некоректні введення) та дано відповіді на всі контрольні запитання методичних вказівок.